

Salesforce Identity & Access Management — Concepts Primer

A plain-language learning reference to build real intuition before we solution the Bedrock scenario. Every concept has a **"Plain-English"** explanation and often an everyday analogy. Nothing here is a design decision yet — this is about *understanding*.

0. The one mental model to hold

Almost every identity topic is really one of two questions:

- **Authentication (AuthN)** = "Who are you?" — proving you are who you claim to be. This is *logging in*.
- **Authorization (AuthZ)** = "What are you allowed to do?" — what you can see or do once you're in, often *on someone else's behalf*.

Analogy — an airport: - **Authentication** is the passport check: they confirm *you are you*. - **Authorization** is your boarding pass: it says *which flight and seat* you're allowed into. You can be perfectly authenticated (valid passport) and still not be authorized for the first-class lounge.

Keep them separate in your head — mixing them up is the #1 source of confusion.

The cast of characters (actors)

There are only ever a handful of players. Learn these names and 80% of the jargon unlocks:

- **User** (also "resource owner" or "principal") — the human.
- **Identity Provider (IdP)** — the trusted authority that *checks* who you are and then *vouches* for you. **Analogy: the passport office.** It issues a document others trust.
- **Service Provider (SP) / Relying Party (RP)** — the app you're trying to use, which *trusts* the IdP's word. **Analogy: a foreign country that accepts your passport** instead of re-verifying your birth certificate.
- **Resource Server** — where the actual protected data or API lives.
- **Client / Connected App** — a piece of software that wants to access the resource server.

Why Salesforce is confusing at first: it can play *every one* of these roles — sometimes several in the same flow. It can be the SP (you log *into* Salesforce), the IdP (Salesforce logs you *into* other apps), the resource server (external apps call Salesforce's API), and the client (Salesforce calls someone else's API). Whenever you're stuck, ask: "Which role is Salesforce playing right now, and which way does the trust point?"

1. My Domain (the foundation everything sits on)

What it is: a unique, branded web address for your Salesforce org, like `bedrock.my.salesforce.com`.

Plain-English: it's your company's own **branded front door** to Salesforce, instead of a generic shared entrance. Once you have your own front door, you get to decide what the door looks like and who's allowed to knock.

Why it matters so much — it's the control point for almost every identity feature: - It hosts the **login page** you can brand (logo, colours, custom background). - It lets you choose **which login options appear** on that page (password, "Log in with Google," corporate SSO button...). - It is the **address other systems federate with** —

when an external identity provider or app needs to talk to your org, they point at your My Domain URLs. - It enables **Login Discovery** — instead of showing a password box, the page first asks "*what's your email?*" and then routes you to the *right* way to log in (your company's SSO, a social provider, etc.). **Analogy: a smart concierge who looks at your email domain and sends you to the correct check-in desk.**

Exam phrasing: "the role of My Domain" = it's the front door + the identity endpoint that makes branding, SSO, and login-routing possible. Without it, most of the other features can't be switched on.

2. Authentication mechanisms (all the ways to log in)

2.1 Username / password (local login)

Salesforce itself stores your password (hashed) and checks it.

Plain-English: the simplest kind — the app keeps its own list of who's who. Fine for small setups, but in a big company it means *yet another password* and no central control. If someone leaves the company, an admin has to remember to disable this specific account.

2.2 Single Sign-On (SSO) — log in once, get into many

You authenticate **once** with a trusted IdP, and then get into lots of different apps without typing your password again.

Plain-English / analogy: think of a **theme park wristband**. You prove your identity once at the entrance (the IdP), get a wristband, and then every ride (each app/SP) just checks the wristband instead of re-verifying who you are.

There are two main "languages" for doing SSO:

SAML 2.0 — the enterprise workhorse

- Uses **XML documents called "assertions."** An assertion is basically a **signed, notarized letter** that says "I, the passport office, certify this is Alice, and here are her details."
- The letter is **digitally signed** so the receiver can prove it's genuine and wasn't tampered with (trust is set up in advance by exchanging certificates).
- **Salesforce is usually the SP here.** Flow in plain terms: 1. Alice tries to open Salesforce. 2. Salesforce says "I don't authenticate you myself — go see your company's IdP" and **redirects her browser** there. 3. The corporate IdP (Microsoft Entra ID, AD FS, Ping...) checks Alice against **Active Directory**. 4. The IdP writes the signed letter (assertion) and sends Alice's **browser** back to Salesforce's special **ACS (Assertion Consumer Service)** URL, carrying the letter. 5. Salesforce checks the signature, reads "this is Alice," and logs her in.
- **SP-initiated** = the journey starts at Salesforce (most common). **IdP-initiated** = the journey starts at the company portal, where Alice clicks a Salesforce tile.

OpenID Connect (OIDC) — the modern one

- Same idea, but built on **OAuth 2.0** and uses lightweight **JSON tokens (JWTs)** instead of bulky XML.
- The "notarized letter" here is called an **ID token** — a compact, signed digital ID card.
- Friendlier for **mobile apps and APIs**, and it's what **social logins** (Google, Facebook, LinkedIn) use.

SAML vs OIDC in one line: same purpose (SSO), different era and format — SAML is XML and enterprise-classic; OIDC is JSON/JWT and mobile/web-modern.

The browser's role (why the exam keeps asking)

In these redirect-based flows, the **browser is the trusted courier**. It: - carries the **redirects** back and forth between the app and the IdP, - holds the user's **IdP session cookie** (which is *why* the second app doesn't ask you to log in again — SSO magic), - **delivers the signed assertion/token** to the app.

Being able to narrate "at this step the browser gets redirected here, carrying this" is exactly the 20%-weighted skill.

2.3 Delegated Authentication

Salesforce shows *its own* login box, but when you type your password it secretly **passes it to an external service** that answers "yes/no."

Plain-English: the bouncer at the door still takes your ID, but instead of checking it himself he phones head office to confirm. Older approach; weaker because the password actually travels through Salesforce. Usually a fallback, not a first choice.

2.4 Social sign-on & Authentication Providers

An **Auth. Provider** is a Salesforce setting that lets people log in using an outside account — Google, Facebook, LinkedIn, Apple, Microsoft, or a **custom** provider.

The clever bit: the first time someone logs in this way, a **Registration Handler** (a small Apex class you write) runs and **creates or updates their Salesforce user automatically** and links the two identities together (a "Third-Party Account Link").

Plain-English / analogy: like a nightclub that lets you in with your **driver's licence from another state**. The first time, the club quickly makes you a member card (the Registration Handler) and links it to your licence, so next time it's instant.

2.5 Login Flows

A **Login Flow** is a step that runs **after** your password/SSO is verified but **before** you're actually let in.

Plain-English: it's the "**one more thing before you enter**" screen. Use it to make someone accept **Terms of Service**, answer a security question, register a phone number, show a compliance notice, or force extra verification.

Analogy: the waiver form you sign after they've confirmed your booking but before they hand you the keys.

2.6 MFA & Identity Verification

Multi-Factor Authentication = proving identity with **two different kinds** of evidence, so a stolen password alone isn't enough.

The classic three "factors": - **Something you know** (password), - **Something you have** (your phone, a security key), - **Something you are** (fingerprint/face).

Salesforce verification methods include: - **Authenticator app** (rotating 6-digit code / push approval), - **Security key / passkey** (WebAuthn/FIDO2) — the **strongest**, because it's **phishing-resistant** (a fake site can't reuse it), - **SMS or email one-time code (OTP)** — the **weakest** of the three (SIM-swap/interception risk) but **easy and mobile-friendly**, which is why it's popular for consumer/mobile apps.

Step-up / High Assurance: you can require MFA only when someone tries to reach something **sensitive** ("you're logged in, but to open *this* you must re-verify"). **Analogy:** **you're in the hotel, but the vault room needs a second key.**

2.7 Passwordless & Headless Identity

- **Passwordless login:** no password at all — you prove yourself with a **one-time code, magic link, or passkey**. Fewer passwords = fewer things to steal or phish.

- **Headless Identity APIs:** Salesforce exposes login, registration, OTP, password reset, etc. as **raw APIs**, so a company can build a **completely custom-looking website or app** while Salesforce quietly remains the **real identity system** behind the scenes.
- **Plain-English / analogy:** "**headless**" means *no standard face/UI is used*. It's like a restaurant where you design your own dining room and menus, but the **kitchen (Salesforce) is still doing the actual cooking**. Customers never see the kitchen; you control 100% of what they see.

3. Direction of trust: Salesforce as SP vs Salesforce as IdP

This trips up a lot of people, so it gets its own section.

- **Salesforce as the SP (inbound):** users log **into Salesforce** using an outside IdP. *Example: Bedrock employees sign into Salesforce with their corporate AD accounts.* Trust points **from Salesforce -> the corporate IdP**.
- **Salesforce as the IdP (outbound):** Salesforce **logs users into other applications**. Salesforce issues the signed assertion/token, and the downstream app (say, a Document Management System) trusts it — so **that app never needs its own username/password for the user**. Trust points **from the other app -> Salesforce**.

Analogy: - As **SP**, Salesforce is a *country accepting your passport*. - As **IdP**, Salesforce is the *passport office* issuing passports other countries accept.

Same protocols (SAML/OIDC), **opposite direction**. When a question involves two systems, always pin down *who is vouching for whom*.

4. Authorization & OAuth 2.0 (access without sharing passwords)

SSO gets a *person logged in*. **OAuth 2.0** solves a different problem: letting **one app use another app's API on your behalf — without giving it your password**.

The killer analogy — the valet key: your car has a **valet key** that can start the engine and drive, but **can't open the glovebox or trunk**. You hand the valet limited power without giving them your full keychain. **OAuth access tokens** are valet keys: scoped, limited, and revocable. You never hand over your actual password.

4.1 The OAuth roles

- **Resource Owner** — you, the user who owns the data.
- **Client** — the app asking for access (defined in Salesforce as a **Connected App / External Client App**).
- **Authorization Server** — the one who issues tokens (can be Salesforce or an external service).
- **Resource Server** — the API holding the protected data.

4.2 The tokens (and what each is for)

- **Authorization code** — a short-lived, one-time **claim ticket** handed back through the browser, which the app then swaps (server-to-server) for real tokens. Why the extra step? So the powerful tokens never ride around in the browser where they could be stolen.
- **Access token** — the **valet key** you actually present to the API. Short-lived on purpose (if stolen, it expires fast).
- **Refresh token** — a "**get a new valet key**" coupon. Long-lived; lets the app get fresh access tokens without bugging you to log in again. **Revoking this coupon cuts off the app** — remember this for the "lost device" requirement.
- **ID token** — (OIDC) the signed **digital ID card** describing *who you are* (that's authentication info riding along).

- **Scopes** — the list of what the valet key is allowed to do (e.g. read your profile, call the API, get a refresh token). You may be shown a **consent screen** to approve them. **Analogy: the app asking "may I see your contacts and post on your behalf?" and you ticking the boxes.**

4.3 Connected Apps (the rulebook for outside apps)

A **Connected App** is the Salesforce record that defines **how an external app is allowed to integrate** via OAuth/SAML: - its identity (consumer key/secret or certificate), - which **scopes** it may request, - which **OAuth flows** it may use, - **who's allowed** to use it (pre-authorize by profile/permission set), - security policies (IP ranges, session timeouts, refresh-token lifetime).

Plain-English / analogy: it's the **vendor badge agreement**. Before a contractor's app can walk around your building, you issue it a badge that says exactly which doors it can open, during which hours, and who signed off. It's the **governance boundary for external parties**.

4.4 The OAuth flows — and the crucial split

The exam explicitly separates **"a human is present and consents"** from **"no human, system-to-system."** Here's the plain-English version:

Flows WITH a user present (interactive):

Flow	Plain-English	When
Web Server (Authorization Code) + PKCE	User logs in and consents in the browser; the app grabs a claim ticket and swaps it for tokens on its server . PKCE adds a secret handshake so a stolen ticket is useless to a thief.	The modern default for web & mobile apps.
User-Agent	Older style where the token is handed straight to the browser/SPA.	Legacy; now replaced by Auth-Code + PKCE.
Device	You see a code on a TV/kiosk and approve it from your phone.	Devices with no real keyboard.
Refresh Token	Not a login — silently trades the "coupon" for a new access token.	Keeping someone logged in after the first consent.

Flows with NO user (server-to-server / machine-to-machine):

Flow	Plain-English	When
JWT Bearer	A backend proves itself by presenting a certificate-signed digital badge (JWT) — no password, no human, no consent screen.	Trusted, automated system-to-system integration.
Client Credentials	A backend uses its own id/secret to get a token for its own access (runs as a set "integration user").	Simple system-to-system where the app acts as itself.
SAML Bearer	Uses an existing SAML assertion to obtain an OAuth access token.	Bridging SAML SSO into API access.

Anti-pattern to name-drop: the **Username-Password flow** exists but is **discouraged** because the app handles the real password directly — the opposite of the valet-key idea.

The mental shortcut: *human in the loop* -> Authorization Code + PKCE. No human, just servers -> JWT Bearer (or Client Credentials).

4.5 Named Credentials & External Credentials (when Salesforce is the one calling out)

When **Salesforce needs to call an external API** (a credit bureau, an SMS gateway, a document system), you must **not** hard-code passwords/keys in your code. **Named Credentials** (the endpoint) + **External Credentials** (the authentication) let the **platform manage the login handshake and tokens for you**.

- **Named principal** — everyone shares **one** service identity (classic server-to-server).
- **Per-user** — each user calls out with **their own** external token (delegated access).

Plain-English / analogy: it's like your **company's expense system that already knows the vendor logins** — you click "order," and it authenticates on your behalf. You (the developer/user) **never see or store the secret**. This is the clean answer whenever a requirement says "credentials must never be exposed or stored."

5. Identity lifecycle management (joiners, movers, leavers)

This answers "*who gets an account, how it stays in sync, and how it's shut off when someone leaves*" — a heavily weighted (15%) topic.

- **Just-In-Time (JIT) provisioning** — the Salesforce user is **created or updated automatically the first time they log in via SSO**, using the details in the assertion/token.
- **Plain-English / analogy:** the account is made **the moment you first show up at the door**, using the info on your passport. No pre-registration needed.

Limitation: JIT is *lazy* — it only acts **at login**. It can create and update, but it **can't disable someone who never logs in again** (i.e. it can't offboard).

SCIM (System for Cross-domain Identity Management) — a standard **REST API** that lets the IdP/HR system **actively push** user create/update/disable into Salesforce.

- **Plain-English / analogy:** HR pushes the new hire's record into every system on day one, and flips it off the day they leave — proactively, without waiting for the person to log in.

This is what makes **automatic deprovisioning** possible.

Identity Connect — a Salesforce component that **syncs on-premise Active Directory** -> **Salesforce** on a schedule or near-real-time. (Cloud-native alternative: Entra ID's provisioning to Salesforce.)

Plain-English: a **bridge/sync-robot** that keeps Salesforce's user list mirroring the on-prem AD.

Deprovisioning & session/token revocation — turning off the source identity should **freeze the Salesforce user and end their live sessions and tokens**. For a **lost or stolen phone**, you don't necessarily fire the employee — you **revoke that device's OAuth tokens (the refresh "coupon") and kill active sessions**, so the phone becomes useless while the person keeps their account.

- **Analogy:** deactivating a **lost hotel keycard** without evicting the guest.

JIT vs SCIM in one line: **JIT** creates accounts *lazily, at login*; **SCIM** provisions *proactively and can also deprovision*. So "access must be removed automatically when someone leaves" -> you need SCIM-style push, not JIT alone.

6. External identity / CIAM (customers & partners)

"CIAM" = **Customer Identity & Access Management** — identity for millions of *external* people, not employees.

- **Experience Cloud** hosts external users (customers, partners) with special license types (Customer Community, Partner Community, **External Identity**).
 - **Analogy:** the **guest wing** of the hotel — separate entrance and amenities from the staff areas.
 - **Bring-Your-Own-IdP (BYOIdP) per partner** — each partner company logs its users in with **its own** corporate identity provider. Salesforce can trust **many IdPs at once** and uses **Login Discovery** to send each partner's users to the right one based on their email domain.
 - **Analogy:** a shared office building where **each tenant company uses its own badge system**, and the lobby concierge routes you to your employer's reader based on who you work for.
 - **Registration Handler** — the Apex logic that decides how a brand-new external/social user is turned into a Contact/Person Account + user on the fly.
 - **Identity linking** — connecting an external identity (PayPal, Google, etc.) to a Salesforce user record via a **Third-Party Account Link**, so "the PayPal person" and "the Salesforce person" are known to be the same.
-

7. Trust, cryptography & session security (the plumbing that makes it safe)

- **TLS** — the padlock (`https`) that **encrypts data in transit** so no one can eavesdrop. **mTLS** = *both* sides show certificates and prove who they are (mutual).
 - **Signing & validation** — SAML assertions and JWTs are **digitally signed**; the receiver checks the signature with the sender's **public certificate**.
 - **Analogy:** a **wax seal / notary stamp**. Anyone can verify it's genuine and unbroken, but no forger can reproduce it. This is what "verifiable protocols that prevent tampering" means.
 - **Certificate & Key Management** — where Salesforce keeps the certificates/keys used to sign assertions and JWTs.
 - **Session security** — timeouts, **High Assurance** requirements for sensitive areas, locking a session to an IP, allowed **login IP ranges**, and revoking sessions.
 - **Central credential governance** — for staff, you often **turn off Salesforce's own "forgot password"/reset** so that *all* password changes and recovery go through the **corporate IdP** — one single source of truth, no side doors.
-

8. Standards glossary (quick recall)

Term	One-liner
LDAP / AD	The directory (and protocol) where a company's employee identities/passwords live.
SAML 2.0	XML, signed assertions , enterprise SSO (Salesforce as SP or IdP).
OAuth 2.0	Delegated authorization — valet keys (access tokens), no password sharing.

OIDC	Identity layer on top of OAuth; issues a signed ID token (JWT).
JWT	A compact, signed JSON token — used as the ID token and in the JWT-bearer flow.
PKCE	A secret-handshake add-on that stops a stolen authorization code from being used.
SCIM	REST standard to push user create/update/disable across systems.
MFA / TOTP / WebAuthn	Second factors; passkeys/security keys are phishing-resistant , SMS OTP is convenient but weakest.
ACS	The SAML endpoint on the SP that receives (consumes) the assertion.
Named Credential	Salesforce-managed endpoint + stored-safely auth for outbound callouts.
IdP-initiated / SP-initiated	Did the login journey start at the identity provider or at the app?

9. How these concepts pre-map to the Bedrock requirements (orientation only)

Concept	Likely relevant Bedrock requirement(s)
SAML SSO, Salesforce as SP , My Domain, browser redirects	R1 internal AD SSO; F1 authentication flow
MFA / SMS OTP, High Assurance, mobile	R3 branded mobile app + strong MFA
SCIM + Identity Connect, JIT, deprovisioning, token/session revoke	R2 auto provisioning/deprovisioning; R9 lost device
Suppress native reset, central recovery	R4 central credential governance
Auth Providers + Registration Handler (social)	R5 social sign-up
Named Credential + secure callout, login gating	R6 real-time credit check
Login Flow	R11 one-time Terms of Service consent
Salesforce as IdP (SAML/OIDC), no stored creds	R8 DMS access
Multiple external IdPs, Login Discovery, Experience Cloud	R12 partner BYOldP federation
OAuth (authN + authZ), identity linking, connected app	R13 PayPal in one session

Headless Identity APIs, passwordless, SF as authoritative IdP	R14 branded headless portal
Connected Apps, JWT-bearer vs Auth-Code+PKCE	F2 user-interactive authZ; F3 server-to-server authZ; F4 external parties

(This mapping is only to orient you — we'll design each properly, requirement by requirement, later.)

10. Fast revision — the 12 things to be able to say out loud

1. **AuthN** = who you are; **AuthZ** = what you may do (airport: passport vs boarding pass).
2. **My Domain** = branded front door + identity endpoint; prerequisite for everything.
3. **SSO** = prove once, enter many (theme-park wristband).
4. **SAML** = signed XML letter, enterprise; **OIDC** = signed JSON ID card, modern/mobile.
5. The **browser** is the courier carrying redirects + assertions in SSO.
6. **SP** = Salesforce trusts an outside IdP to log users *in*; **IdP** = Salesforce logs users *into* other apps (so they store no creds).
7. **OAuth** = valet key: limited, revocable access **without** sharing your password.
8. **Access token** = short-lived valet key; **refresh token** = coupon for a new one (revoke it to cut off a lost device).
9. **Connected App** = the vendor-badge rulebook for an outside app (scopes, flows, who's allowed).
10. **Human present** -> **Auth-Code + PKCE**; **no human** -> **JWT Bearer / Client Credentials**.
11. **JIT** provisions lazily at login; **SCIM** pushes proactively and can **deprovision** (offboarding).
12. **Named Credentials** = Salesforce calls external APIs without you ever storing the secret.